

Rockchip Clock Developer Guide

ID: RK-KF-YF-173

Release Version: V1.2.0

Release Date: 2024-08-02

Security Level: ☐Top-Secret ☐Secret ☐Internal ☒Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED "AS IS". ROCKCHIP ELECTRONICS CO., LTD.("ROCKCHIP")DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip's registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2025. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document mainly introduces the framework of the RK platform clock subsystem and its configuration.

Product Version

Chipset	Kernel Version
RK303X	LINUX4.4 & 4.19 & 5.10
RK312X	LINUX4.4 & 4.19 & 5.10
RK322X	LINUX4.4 & 4.19 & 5.10
RK3288X	LINUX4.4 & 4.19 & 5.10
RK3328	LINUX4.4 & 4.19 & 5.10
RK3368	LINUX4.4 & 4.19 & 5.10
RK3399	LINUX4.4 & 4.19 & 5.10
RV1108	LINUX4.4 & 4.19 & 5.10
PX30	LINUX4.4 & 4.19 & 5.10
RV1126	LINUX4.19 & 5.10
RK356X	LINUX4.19 & 5.10
RK3588	LINUX5.10
RK3576	LINUX6.1
RV1103B	LINUX5.10
RK3506	LINUX6.1

Intended Audience

This document (this guide) is mainly intended for:

Technical support engineers

Software development engineers

Revision History

Version	Author	Date	Change Description
V1.0.0	Zhang Qing	2021-01-18	First temporary version release
V1.0.1	Huang Ying	2021-04-26	Added document identifier, modified format
V1.1.0	Zhang Qing	2021-12-22	Added RK3588, kernel support 5.10
V1.2.0	Zhang Qing	2024-08-02	Added RK3576\RV1103B\RK3506, kernel support 6.1

Table of Contents

Rockchip Clock Developer Guide

1. Solution Overview
 - 1.1 Overview
 - 1.2 Important Concepts
 - 1.3 Clocking Scheme
 - 1.4 Overall Process
 - 1.5 Code Structure
2. CLOCK Development Guide
 - 2.1 Overview
 - 2.2 Concepts Related to Clocks
 - 2.2.1 PLL
 - 2.2.2 Bus Architecture
 - 2.2.3 GATING
 - 2.3 Clock Configuration
 - 2.3.1 Clock Initialization Configuration
 - 2.3.2 Clock ID
 - 2.3.3 Main CLOCK Register Types Functions
 - 2.3.4 Clock Configuration for Driver
 - 2.4 CLOCK API Interface
 - 2.4.1 Main CLOCK API
 - 2.4.2 Example
 - 2.5 CLOCK Debugging
3. Common Problem Analysis
 - 3.1 PLL Configuration
 - 3.1.1 PLL Frequency Table Definition
 - 3.1.2 PLL Calculation Formula
 - 3.2 Configuration of Specific Clocks
 - 3.2.1 LCDDC Display-Related Clocks
 - 3.2.2 Fractional Frequency Division
 - 3.2.3 Ethernet Clock
 - 3.2.4 PLL Parameters for Multi-Path Clock Input

Figure Catalog

[*Figure 1-1 Example of Clock Tree*](#)

[*Figure 1-2 Clock Configuration Flowchart*](#)

[*Figure 1-3 CLOCK Code Structure*](#)

[*Figure 2-1 PLL Description*](#)

[*Figure 2-2 Bus Clock Structure*](#)

[*Figure 2-3 GATING Example Diagram*](#)

[*Figure 2-4 CLOCKID Example Diagram*](#)

[*Figure 2-5 CLOKTYPE Example Diagram*](#)

[*Figure 2-6 CLOCKPARENT Example Diagram*](#)

[*Figure 2-7 TESTCLOCK Example Diagram*](#)

[*Figure 2-8 TESTCLOCK MUX*](#)

[*Figure 2-9 TESTCLOCK DIV*](#)

[*Figure 2-10 TESTCLOCK GATE*](#)

[*Figure 3-1 DCLK Allocation Diagram*](#)

[*Figure 3-2 Fractional Frequency Division Clock Schematic*](#)

1. Solution Overview

1.1 Overview

This section primarily outlines the key concepts, clock schemes, overall processes, and code structure related to the clock subsystem.

1.2 Important Concepts

Clock Subsystem

The clocks discussed here are the hierarchical framework that provides clocks to various components of the SOC, not the time used by the kernel. Like other modules, CLOCK also has a framework to adapt to different platforms. Above the adaptation layer are customer code and interfaces, which are the drivers for various modules (such as peripherals that require clock signals, USB, etc.). Below the adaptation layer are the specific clock operation details of the SOC platform.

Clock Tree Structure

Mainstream processor platforms that can run LINUX all have a very complex CLOCK TREE. If we take a processor's SPEC and look at the section related to CLOCK, there will definitely be a very large and complex tree diagram. This diagram is composed of CLOCK-related devices and the clocks output by these devices.

Related Devices

CLOCK-related devices include: Oscillators (active oscillators, also known as harmonic oscillators) or Crystals (passive oscillators, also known as crystal oscillators) for generating CLOCK; PLLs (Phase Locked Loops) for frequency multiplication; Dividers for frequency division; MUXes for multiplexing; AND gates for CLOCK ENABLE control; hardware modules that use CLOCK (can be referred to as CONSUMERS); and so on.

1.3 Clocking Scheme

Each SOC has its own clock distribution scheme, which mainly includes the settings of PLL, the parent attributes of various CLOCKS, DIV, MUX, etc. The clock schemes vary with different chips.

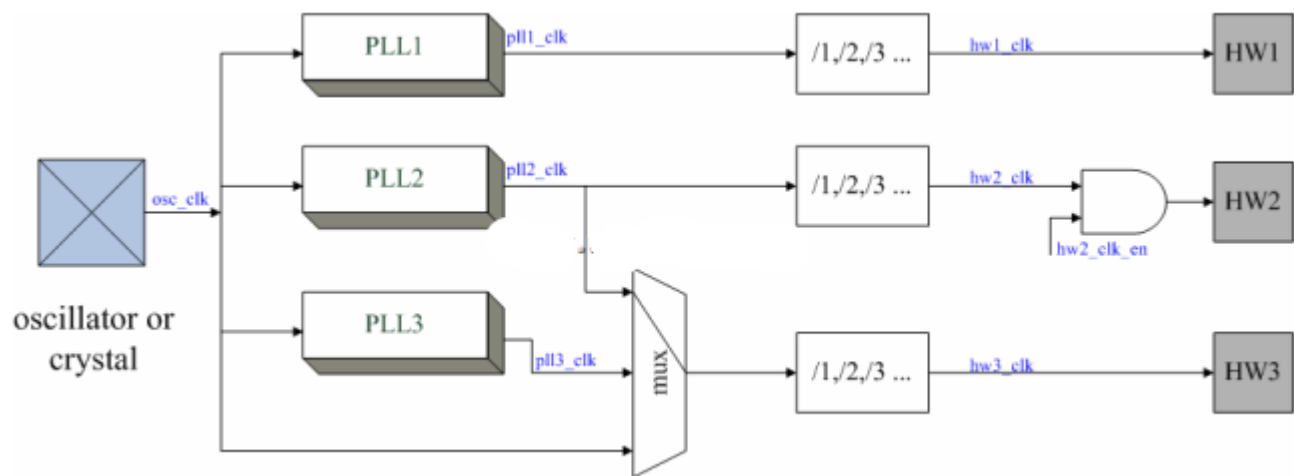


Figure 1-1: Example diagram of a clock tree

1.4 Overall Process

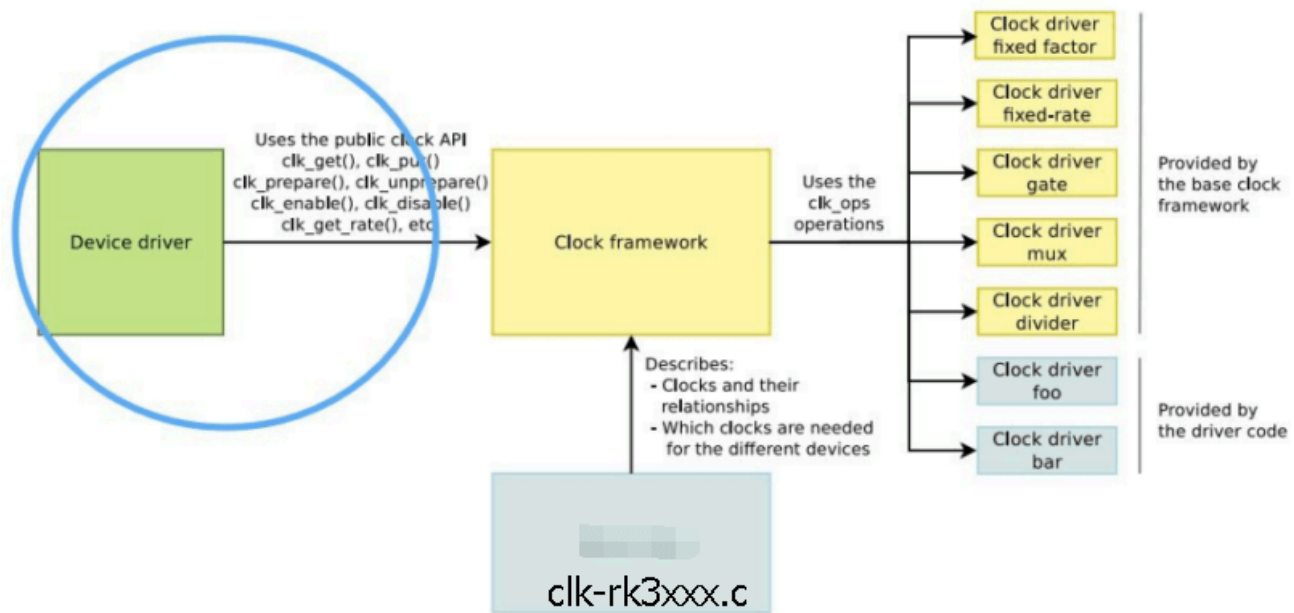


Figure 1-2: Clock Configuration Flowchart

The main content includes (not all CLOCKS need to be supported):

- 1. Enable/Disable CLOCK.
- 2. Set the frequency of the CLOCK.
- 3. Select the Parent of the CLOCK.

1.5 Code Structure

The software framework of CLOCK consists of the CLOCK Device Tree (register description of CLOCK, tree-like relationships between CLOCKS, etc.), Device driver configuration for CLOCK, and CLOCK API. The functions of these three parts and the paths of CLOCK code are shown in Table 1-1.

Item	Function	Path
clk-rk3xxx.c	Mainly contains the register description of CLOCK and the tree-like relationship description between CLOCKS	drivers/clk/rockchip/clk-rk3xxx.c
rk3xxx-cru.h	Contains the ID definitions of CLOCK, matching CLOCK Name through ID.	include/dt-bindings/clock/rk3xxx-cru.h include/dt-bindings/clock/rockchip,rk3xxx-cru.h
RK Special Handling	Handles RK's PLL clocks, and some special clocks of RK such as LCDC, I2S, etc.	drivers/clk/rockchip/clk-xxx.c
CLK API	Provides interfaces for Drivers to call in the Linux environment	drivers/clk/clk-xxx.x

Figure 1-3: Structure of CLOCK Code

2. CLOCK Development Guide

2.1 Overview

This section describes how to modify clock configurations, use API interfaces, and debug CLOCK programs.

2.2 Concepts Related to Clocks

2.2.1 PLL

Phase-Locked Loop, which is fed by a 24M crystal oscillator, and then the internal PLL locks to the corresponding frequency. This is the clock source for all CLOCKS of the SOC. All bus and device clocks of the SOC are derived from the PLL frequency division. The main PLLs on the RK platform are:

PLL	Sub-device	Purpose	Remarks
APLL	CLOCK_CORE	CPU clock	Generally used only for the CPU, as the CPU will change frequency, APLL will vary according to the CPU's frequency requirements
DPLL	CLOCK_DDR	DDR clock	Generally used only for DDR, as DDR will change frequency, DPLL will vary according to DDR's requirements
GPLL		Provides bus and peripheral clock backup	Generally set at 594M or 1200M, ensuring basic 100, 200, 300, 400M clock outputs
CPLL		GMAC or other devices backup	Generally might be 400, 500, 800, 1000M. Or exclusively used for Lcdc.
NPLL		Backup for other devices	Generally might be 1188M, or exclusively used for Lcdc.

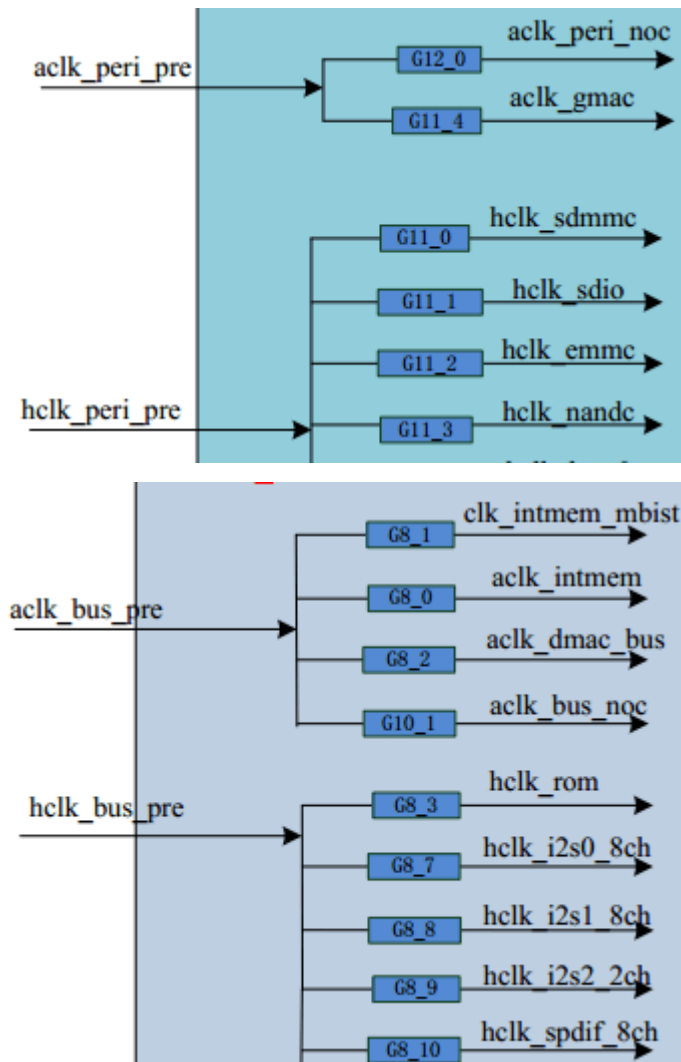
Figure 2-1 PLL Description

2.2.2 Bus Architecture

The bus architecture of our SOC includes ACLK_PERI, HCLK_PERI, PCLK_PERI, ACLK_BUS, HCLK_BUS, and PCLK_BUS. (ACLK is used for data transfer, while PCLK and HCLK are generally used for register read and write operations).

The distinction between BUS and PERI is primarily to differentiate between high-speed and low-speed buses. The ACLK range is 100-300M, PCLK ranges from 50M to 150M, and HCLK ranges from 37M to 150M. Devices under BUS are mainly low-speed devices such as I2C, I2S, SPI, etc., while devices under PERI are typically EMMC, GMAC, USB, etc. There will be some differences in the design of different chips. For example: for certain devices that require higher bus speeds, a separate independent ACLK may be designed for this device (such as ACLK_EMMC or ACLK_USB, etc.).

The bus clocks for each device will be hung under these clocks, as shown in the following structure:



pd_perilp	cm0, crypto, dcf, imem, dmac, bootrom, efuse_con, spi, i2c, uart, saradc, tsadc
pd_perihp	pcie, usb2, hsic

Figure 2-2 Bus Clock Structure

(For example: GMAC wants to improve the bus frequency of its own device to achieve its fast data copy or transfer, it can be achieved by increasing ACLK_PERI)

Note:

The design on RK3399 completely separates high-speed and low-speed buses, divided into high-speed: ACLK_PERIHP, HCLK_PERIHP, PCLK_PERIHP; low-speed: ACLK_PERILP0, HCLK_PERILP0, PCLK_PERILP0, HCLK_PERILP1, PCLK_PERILP1. This is done for optimal power consumption, and different bus frequencies can be set according to different needs. (For details on which device is under which bus, see the clock diagram) Reference can be made (EMMC, GMAC, USB, etc., have their own ACLK).

2.2.3 GATING

In the CLOCK framework, there are numerous GATING mechanisms primarily utilized to reduce power consumption. When certain devices are powered off and there is no need to maintain the CLOCK, GATING can be turned off to conserve energy.

The GATING within the RK CLOCK framework is structured like a tree, with parent-child attributes. The GATING switch employs a reference counting mechanism, which is used to traverse and open its parent CLOCK when a CLOCK is turned on. When a child CLOCK is shut down, the parent CLOCK will traverse all its child CLOCKS, and only shut down the parent CLOCK when all children are closed.

(For example: When I2S2 is in use, it is necessary to open the three GATINGs shown in the diagram (as shown in Figure 2-2), but the software only needs to open the last-level GATING, and the clock structure will automatically open its parent GATING.)

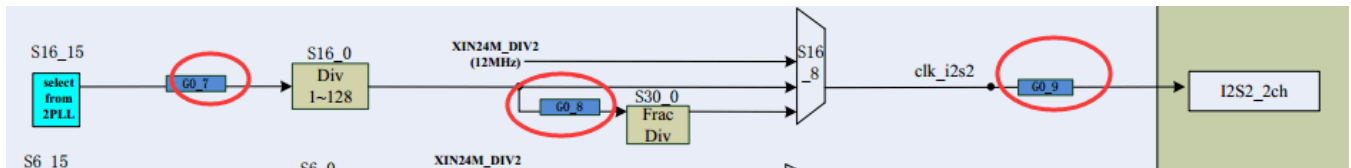


Figure 2-3: GATING Example Diagram

2.3 Clock Configuration

2.3.1 Clock Initialization Configuration

Unlike LINUX3.10, after the 4.4 kernel, clock initialization uses "of_clk_set_defaults" and then parses "assigned-clocks" to obtain the CLOCK ID, then acquires the CLOCK Name, parses "assigned-clock-parents" to get the PARENT that needs to be set, and parses "assigned-clock-rates" to get the frequency that needs to be set.

```
arch/arm64/dts/rockchip/rk33xx.dtsi

cru: clock-controller@ff760000 {
    compatible = "rockchip,rk3399-cru";
    reg = <0x0 0xff760000 0x0 0x1000>;
    #clock-cells = <1>;
    #reset-cells = <1>;
    assigned-clocks =
        <&cru ACLK_VOP0>, <&cru HCLK_VOP0>,
        <&cru ACLK_VOP1>, <&cru HCLK_VOP1>,
        <&cru ARMCLKL>, <&cru ARMCLKB>,
        <&cru PLL_GPLL>, <&cru PLL_CPLL>,
        <&cru PLL_NPLL>,
        <&cru ACLK_PERIHP>, <&cru HCLK_PERIHP>,
        <&cru PCLK_PERIHP>,
        <&cru ACLK_PERILP0>, <&cru HCLK_PERILP0>,
        <&cru PCLK_PERILP0>,
        <&cru HCLK_PERILP1>, <&cru PCLK_PERILP1>;
};
```

1. Frequency

Frequencies set during the initialization of the CLOCK TREE:

```

assigned-clock-rates =
    <400000000>, <200000000>,
    <400000000>, <200000000>,
    <816000000>, <816000000>,
    <594000000>, <800000000>,
    <1000000000>,
    <150000000>, <75000000>,
    <37500000>,
    <100000000>, <100000000>,
    <50000000>,
    <100000000>, <50000000>;

```

2. Parent

Parents set during the initialization of the CLOCK TREE:

```

assigned-clock-parents =
    <&cru VPLL>, <&cru VPLL>,
    <&cru CPLL>, <&cru CPLL>,
    <&cru APPLL>, <&cru APLLB>,
    <&cru GPLL>, <&cru GPLL>,
    <&cru GPLL>,
    <&cru GPLL>, <&cru GPLL>;

```

Note:

When configuring Parents and Rates with Assigned, they need to correspond one-to-one with Assigned-Clocks, as the settings are searched and set according to the CLK ID of Assigned-Clocks.

3. Gating

Whether the CLOCK TREE is enabled by default during initialization:

Note: For clocks that are not enabled by default and are not referenced to be enabled by devices, they will be closed after the CLOCK initialization is completed.

(1) Always-on CLOCKS:

Add Critical configurations in clk-rk3xxx.c, mainly in rk3399_cru_critical_clocks, to add the names of CLOCKS that need to be turned on by default. Once the count of the added CLOCK is incremented by 1, this CLOCK will not be able to be turned off later.

```
static const char *const rk3399_cru_critical_clocks[] __initconst = {
    "aclk_usb3_noc",
    "aclk_gmac_noc",
    "pclk_gmac_noc",
    "pclk_center_main_noc",
    "aclk_cci_noc0",
    "aclk_cci_noc1",
    "clk_dbg_noc",
    "hclk_vcodec_noc",
    "aclk_vcodec_noc",
    "hclk_vdu_noc",
    "aclk_vdu_noc",
};
```

(2) When defining CLOCKS, add the Flag attribute CLK_IGNORE_UNUSED, so even if this CLOCK is not used, it will not be closed when CLOCKS are closed at the end. However, the Enable Cnt seen on the CLOCK TREE is still 0, but the CLOCK is on.

```
GATE(PCLK_PMUGRF_PMU, "pclk_pmugrf_pmu", "pclk_pmu_src", CLK_IGNORE_UNUSED,
RK3399_PMU_CLKGATE_CON(1), 1, GFLAGS),
```

(3) After kernel 6.1, there are changes to always-on clks, and there are two ways to handle this:

Keep all clks always on:

```
chosen: chosen {
-       bootargs = "earlycon=uart8250,mmio32,0xfeb50000 console=ttyFIQ0";
+       bootargs = "earlycon=uart8250,mmio32,0xfeb50000 console=ttyFIQ0
clk_gate.always_on=1 pm_domains.always_on=1";
};
```

Specify a particular clock to keep always on, add the Flag attribute CLK_IS_CRITICAL when defining the CLOCK:

```
GATE(CLK_OSC0_PMU1, "clk_osc0_pmu1", "xin24m", CLK_IS_CRITICAL,
RK3576_PMU_CLKGATE_CON(7), 8, GFLAGS),
```

2.3.2 Clock ID

In Linux 4.4 and above, operations on CLOCK are referenced by CLOCK ID. How to obtain the ID?

Find the CLOCK that needs to be controlled in clk-rk3xxx.c (which can be searched by Name):

The diagram illustrates how to find a clock ID in the kernel source. It shows three code snippets with red circles and arrows pointing to specific fields:

- Snippet 1:** `COMPOSITE_FRACMUX(0, "clk_uart4_frac", "clk_uart4_div", CLK_SET_RATE_PARENT, RK3399_PMU_CLKSEL_CON(6), 0, RK3399_PMU_CLKGATE_CON(0), 6, GFLAGS, &rk3399_uart4_pmu_fracmux);`
 - A red circle around `"clk_uart4_div"` is labeled **CLK ID**.
 - A red circle around `"clk_uart4_frac"` is labeled **clkout_name**.
- Snippet 2:** `DIV(PCLK_SRC_PMU, "pclk_pmu_src", "pclk_pmu_src", CLK_IGNORE_UNUSED, RK3399_PMU_CLKSEL_CON(0), 0, 5, DFLAGS);`
 - A red circle around `"pclk_pmu_src"` is labeled **clkout_name**.
 - A red circle around `"pclk_pmu_src"` is labeled **clk_parent_name**.
- Snippet 3:** `GATE(SCLK_TIMER12_PMU, "clk_timer0_pmu", "clk_timer_src_pmu", 0, RK3399_PMU_CLKGATE_CON(0), 3, GFLAGS);`
 - A red circle around `"clk_timer0_pmu"` is labeled **CLK ID**.
 - A red circle around `"clk_timer_src_pmu"` is labeled **clkout_name**.

Figure 2-4 Example of CLOCKID

2.3.3 Main CLOCK Register Types Functions

Commonly used types include the following:

GATE: Describes GATING, which mainly includes CLOCK ID, type, GATING register offset address, BIT position, etc.

MUX: Describes SELECT, which mainly includes CLOCK ID, type, MUX register offset address, BIT position, etc.

COMPOSITE: Describes CLKs with MUX, DIV, GATING, which mainly includes CLOCK ID, type, MUX, DIV, GATING register offset address, BIT position, etc.

```
#define MUX(_id, cname, pnames, f, o, s, w, mf) \
{ \
    .id = _id, \
    .branch_type = branch_mux, \
    .name = cname, \
    .parent_names = pnames, \
    .num_parents = ARRAY_SIZE(pnames), \
    .flags = f, \
    .muxdiv_offset = o, \
    .mux_shift = s, \
    .mux_width = w, \
    .mux_flags = mf, \
    .gate_offset = -1, \
} \

#define DIV(_id, cname, pname, f, o, s, w, df) \
{ \
    .id = _id, \
    .branch_type = branch_divider, \
    .name = cname, \
    .parent_names = (const char *[]){ pname }, \
    .num_parents = 1, \
    .flags = f, \
    .muxdiv_offset = o, \
    .div_shift = s, \
    .div_width = w, \
    .div_flags = df, \
    .gate_offset = -1, \
} \

#define COMPOSITE(_id, cname, pnames, f, mo, ms, mw, mf, ds, dw, \
    df, go, gs, gf) \
{ \
    .id = _id, \
    .branch_type = branch_composite, \
    .name = cname, \
    .parent_names = pnames, \
    .num_parents = ARRAY_SIZE(pnames), \
    .flags = f, \
    .muxdiv_offset = mo, \
    .mux_shift = ms, \
    .mux_width = mw, \
    .mux_flags = mf, \
    .div_shift = ds, \
    .div_width = dw, \
    .div_flags = df, \
    .gate_offset = go, \
    .gate_shift = gs, \
    .gate_flags = gf, \
} ? end COMPOSITE ?
```

Figure 2-5 CLOCKTYPE Example Diagram

Usage in clk-rk3xxx.c, using these CLOCK registration functions to describe the type of this CLOCK, registers, and parent-child relationships, etc.

```

3:  /* usbphy */
4:  GATE(SCLK_USB2PHY0_REF, "clk_usb2phy0_ref", "xin24m", CLK_IGNORE_UNUSED,
5:      RK3399_CLKGATE_CON(6), 5, GFLAGS),
6:  GATE(SCLK_USB2PHY1_REF, "clk_usb2phy1_ref", "xin24m", CLK_IGNORE_UNUSED,
7:      RK3399_CLKGATE_CON(6), 6, GFLAGS),
8:
9:  GATE(0, "clk_usbphy0_480m_src", "clk_usbphy0_480m", CLK_IGNORE_UNUSED,
10:      RK3399_CLKGATE_CON(13), 12, GFLAGS),
11:  GATE(0, "clk_usbphy1_480m_src", "clk_usbphy1_480m", CLK_IGNORE_UNUSED,
12:      RK3399_CLKGATE_CON(13), 12, GFLAGS),
13:  MUX(0, "clk_usbphy_480m", mux_usbphy_480m_p, CLK_IGNORE_UNUSED,
14:      RK3399_CLKSEL_CON(14), 6, 1, MFLAGS),
15:
16:  MUX(0, "upll", mux_pll_src_24m_usbphy480m_p, 0,
17:      RK3399_CLKSEL_CON(14), 15, 1, MFLAGS),
18:
19:  COMPOSITE_NODIV(SCLK_HSIOPHY, "clk_hsiophy", mux_pll_src_cppll_gppll_nppll_usbphy480m_p, 0,
20:      RK3399_CLKSEL_CON(19), 0, 2, MFLAGS,
21:      RK3399_CLKGATE_CON(6), 4, GFLAGS),
22:
23:  COMPOSITE(ACLK_USB3, "aclk_usb3", mux_pll_src_cppll_gppll_nppll_p, 0,
24:      RK3399_CLKSEL_CON(39), 6, 2, MFLAGS, 0, 5, DFLAGS,
25:      RK3399_CLKGATE_CON(12), 0, GFLAGS),

```

Figure 2-6 CLOCKPARENT Example Diagram

2.3.4 Clock Configuration for Driver

1. Acquire CLOCK Pointer

Add CLOCK reference information in the DTS device node (recommended)

```

clocks = <&cru SCLK_TSADC>, <&cru PCLK_TSADC>;
clock-names = "tsadc", "apb_pclk";

```

```

dev->pclk = devm_clk_get(&pdev->dev, "tsadc");
dev->clk = devm_clk_get(&pdev->dev, "apb_pclk");

```

DTS device node does not include CLOCK reference information

Driver code:

```

dev->pclk = devm_clk_get(NULL, "g_p_saradc");
dev->clk = devm_clk_get(NULL, "clk_saradc");

```

2.4 CLOCK API Interface

2.4.1 Main CLOCK API

1. Header File

```
#include <linux/clock.h>

clk_prepare/clk_unprepare
clk_enable/clk_disable
clk_prepare_enable/clk_disable_unprepare
clk_get/clk_put
devm_clk_get/devm_clk_put
clk_get_rate/clk_set_rate
clk_round_rate
```

2. Obtaining CLOCK Pointer

```
struct clk *devm_clk_get(struct device *dev, const char *id) (recommended for use, as it
automatically releases)
struct clk *clk_get(struct device *dev, const char *id)
static inline struct clk *devm_clk_get_optional(struct device *dev, const char *id)
static inline int __must_check devm_clk_bulk_get(struct device *dev, int num_clks,
struct clk_bulk_data *clks) (recommended for use, for bulk acquisition and control)
static inline int __must_check devm_clk_bulk_get_optional(struct device *dev, int
num_clks, struct clk_bulk_data *clks)
static inline int __must_check devm_clk_bulk_get_all(struct device *dev, struct
clk_bulk_data **clks)
```

3. Preparing/Enabling CLOCK

```
int clk_prepare(struct clk *clk)
/*Call before opening the clock, may cause the system to sleep, so put the sleep part
here, and atomic operations should be placed in enable*/
void clk_unprepare(struct clk *clk)
/*The reverse operation of prepare*/
int clk_enable(struct clk *clk)
/*Atomic operation, turn on the clock, this function must return after the actual usable
clock signal is generated*/
void clk_disable(struct clk *clk)
/*Atomic operation, turn off the clock*/
clk_enable/clk_disable, start/stop the clock. No sleep will occur.
clk_prepare/clk_unprepare, preparation before starting the clock/cleanup after stopping
the clock. May cause sleep.
```

The `clk_prepare_enable` / `clk_disable_unprepare`, `clk_prepare_enable` / `clk_disable_unprepare` (or `clk_enable` / `clk_disable`) must be used in pairs to ensure the reference count is correct.

Note:

Explanation of prepare/unprepare, enable/disable:

The essence of these two sets of APIs is to divide the starting/stopping of CLOCK into Atomic and Non-atomic stages to facilitate implementation and invocation. Therefore, the "no sleep/may sleep" mentioned above has two implications: one is to inform the underlying CLOCK Driver to implement operations that may cause sleep in Prepare/Unprepare, and never in Enable/Disable; the other is to remind the upper layer DRIVER using CLOCK that calling Prepare/Unprepare interfaces may cause sleep, and must not be called in Atomic context

(e.g., areas protected by Mutex locks, interrupt disabling, Spinlock locks), while calling Enable/Disable interfaces can be done with confidence.

Additionally, why does CLOCK's Enable/Disable need to sleep? Here is an example: for example, when enabling PLL CLOCK, after starting the PLL, it is necessary to wait for it to stabilize. The stabilization time of the PLL is quite long, and during this period, the CPU should be yielded (the process sleeps), otherwise, it would waste CPU resources.

Finally, why is there a combined clk_prepare_enable/clk_disable_unprepare interface? If the caller can ensure that it is called in a Non-atomic context, it can sequentially call prepare/enable, disable/unprepared, and for simplicity, the Framework encapsulates these two interfaces.

4. Setting CLOCK Frequency

```
int clk_set_rate(struct clk *clk, unsigned long rate) (unit Hz)
```

(A return value less than 0 indicates that setting the CLOCK has failed)

2.4.2 Example

DTS

```
tsadc: tsadc@ff260000 {
    compatible = "rockchip,rk3399-tsadc";
    reg = <0x0 0xff260000 0x0 0x100>;
    interrupts = <GIC_SPI 97 IRQ_TYPE_LEVEL_HIGH>;
    rockchip,grf = <&grf>;
    clocks = <&cru SCLK_TSADC>, <&cru PCLK_TSADC>;
    clock-names = "tsadc", "apb_pclk";
    assigned-clocks = <&cru SCLK_TSADC>;
    assigned-clock-rates = <750000>;
    resets = <&cru SRST_TSADC>;
    reset-names = "tsadc-apb";
    pinctrl-names = "init", "default", "sleep";
    pinctrl-0 = <&otp_gpio>;
    pinctrl-1 = <&otp_out>;
    pinctrl-2 = <&otp_gpio>;
    #thermal-sensor-cells = <1>;
    rockchip,hw-tshut-temp = <95000>;
    status = "disabled";
};
```

Driver code

```
static int rockchip_thermal_probe(struct platform_device *pdev)
{
    .....

    thermal->reset = devm_reset_control_array_get(&pdev->dev, false, false);
    if (IS_ERR(thermal->reset)) {
```



```

        if (PTR_ERR(thermal->reset) != -EPROBE_DEFER)
            dev_err(&pdev->dev, "failed to get tsadc reset lines\n");
        return PTR_ERR(thermal->reset);
    }

    thermal->num_clks = devm_clk_bulk_get_all(&pdev->dev, &thermal->clks);
    if (thermal->num_clks < 1)
        return -ENODEV;

    error = clk_bulk_prepare_enable(thermal->num_clks, thermal->clks);
    if (error) {
        dev_err(&pdev->dev, "failed to prepare enable tsadc bulk clks: %d\n",
            error);
        return error;
    }
    .....
}

static int rockchip_thermal_remove(struct platform_device *pdev)
{
    .....

    clk_bulk_disable_unprepare(thermal->num_clks, thermal->clks);

    .....
}

```

2.5 CLOCK Debugging

1. CLOCK DEBUGS:

Print the current clock tree structure:

```
cat /sys/kernel/debug/clk/clk_summary
```

2. CLOCK Setting Nodes:

Node commands:

```

get rate:
    cat /sys/kernel/debug/clk_saradc/clk_rate
set rate:
    echo 24000000 > /sys/kernel/debug/clk_saradc/clk_rate
Enable clk:
    echo 1 > /sys/kernel/debug/clk_saradc/clk_enable_count
Disable clk:
    echo 0 > /sys/kernel/debug/clk_saradc/clk_enable_count
Enable clk(kernel 6.1):
    echo 1 > /sys/kernel/debug/clk_saradc/clk_prepare_enable
Disable clk(kernel 6.1):
    echo 0 > /sys/kernel/debug/clk_saradc/clk_prepare_enable

```

3. TEST_CLK_OUT Testing:

Some clocks can be output to test_clk_out to directly test the CLOCK output frequency, which is used to confirm whether certain clock waveforms are normal. Configuration method (taking RK3399 as an example):

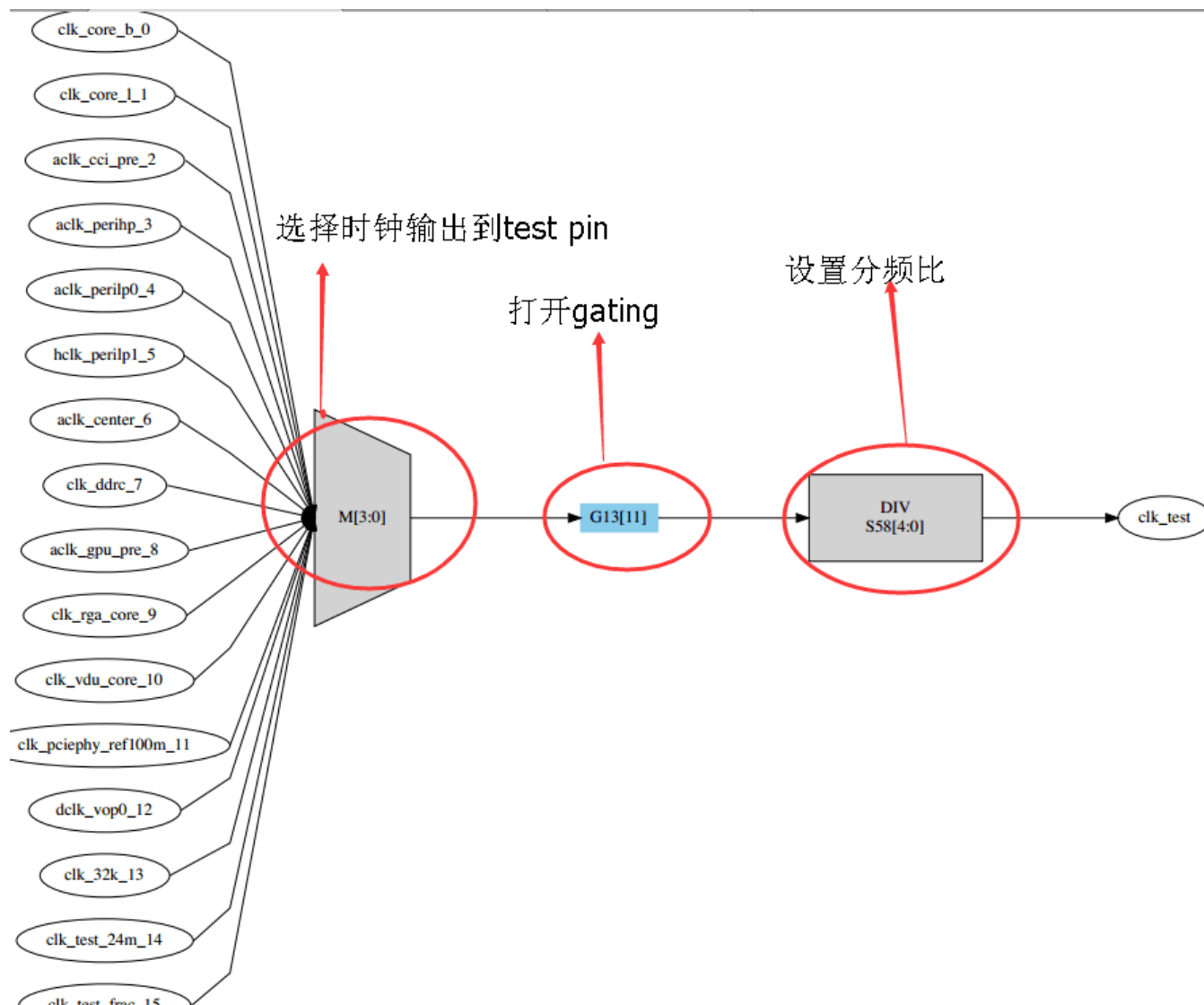


Figure 2-7 TESTCLK Example Diagram

Setting CLOCK's MUX

CRU_MISC_CON

Address: Operational Base + offset (0x050c)

RK3399 TRM

Bit	Attr	Reset Value	Description
3:0	RW	0x0	testclk_sel Output clock selection for test 4'h0: clk_core_b_2wrap 4'h1: clk_core_l_2wrap 4'h2: aclk_cci_2wrap 4'h3: aclk_perihp_2wrap 4'h4: aclk_perilp0_2wrap 4'h5: hclk_perilp1_2wrap 4'h6: aclk_center_2wrap 4'h7: clk_ddrc_2wrap 4'h8: aclk_gpu_2wrap 4'h9: clk_rga_core_2wrap 4'ha: clk_vdu_core_2wrap 4'hb: clk_pciephy_ref100m 4'hc: dclk_vop0_2wrap 4'hd: clk_rtc 4'he: clkout_24m 4'hf: clk_wifi

Figure 2-8 TESTCLOCK_MUX

Setting CLOCK's DIV

CRU_CLKSEL58_CON

Address: Operational Base + offset (0x01e8)

0:3	RW	0x0	reserved
4:0	RW	0x1f	clk_test_div_con test divider control register clk=clk_src/(div_con+1)

Figure 2-9 TESTCLOCK_DIV

Setting CLOCK's GATING

CRU_CLKGATE13_CON

Address: Operational Base + offset (0x0334)

Figure 2-10 TESTCLOCK_GATE

3. Common Problem Analysis

3.1 PLL Configuration

3.1.1 PLL Frequency Table Definition

```
static struct rockchip_pll_rate_table rk3399_pll_rates[] = {
    /* _mhz, _refdiv, _fbdiv, _postdiv1, _postdiv2, _dsmpd, _frac */
    RK3036_PLL_RATE(2208000000, 1, 92, 1, 1, 1, 0),
    RK3036_PLL_RATE(2184000000, 1, 91, 1, 1, 1, 0),
    RK3036_PLL_RATE(2160000000, 1, 90, 1, 1, 1, 0),
    RK3036_PLL_RATE(2136000000, 1, 89, 1, 1, 1, 0),
    RK3036_PLL_RATE(2112000000, 1, 88, 1, 1, 1, 0),
    RK3036_PLL_RATE(2088000000, 1, 87, 1, 1, 1, 0),
    RK3036_PLL_RATE(2064000000, 1, 86, 1, 1, 1, 0),
    RK3036_PLL_RATE(2040000000, 1, 85, 1, 1, 1, 0),
    RK3036_PLL_RATE(2016000000, 1, 84, 1, 1, 1, 0),
    RK3036_PLL_RATE(1992000000, 1, 83, 1, 1, 1, 0),
    RK3036_PLL_RATE(1968000000, 1, 82, 1, 1, 1, 0),
    RK3036_PLL_RATE(1944000000, 1, 81, 1, 1, 1, 0),
    RK3036_PLL_RATE(1920000000, 1, 80, 1, 1, 1, 0),
    RK3036_PLL_RATE(1896000000, 1, 79, 1, 1, 1, 0),
    RK3036_PLL_RATE(1872000000, 1, 78, 1, 1, 1, 0),
    RK3036_PLL_RATE(1848000000, 1, 77, 1, 1, 1, 0),
    RK3036_PLL_RATE(1824000000, 1, 76, 1, 1, 1, 0),
    RK3036_PLL_RATE(1800000000, 1, 75, 1, 1, 1, 0),
    RK3036_PLL_RATE(1776000000, 1, 74, 1, 1, 1, 0),
    RK3036_PLL_RATE(1752000000, 1, 73, 1, 1, 1, 0),
    RK3036_PLL_RATE(1728000000, 1, 72, 1, 1, 1, 0),
    RK3036_PLL_RATE(1704000000, 1, 71, 1, 1, 1, 0),
    RK3036_PLL_RATE(1680000000, 1, 70, 1, 1, 1, 0),
    RK3036_PLL_RATE(1656000000, 1, 69, 1, 1, 1, 0),
    RK3036_PLL_RATE(1632000000, 1, 68, 1, 1, 1, 0),
    RK3036_PLL_RATE(1608000000, 1, 67, 1, 1, 1, 0),
    RK3036_PLL_RATE(1600000000, 3, 200, 1, 1, 1, 0),
    RK3036_PLL_RATE(1584000000, 1, 66, 1, 1, 1, 0),
    RK3036_PLL_RATE(1560000000, 1, 65, 1, 1, 1, 0),
    RK3036_PLL_RATE(1536000000, 1, 64, 1, 1, 1, 0),
    RK3036_PLL_RATE(1512000000, 1, 63, 1, 1, 1, 0),
    RK3036_PLL_RATE(1488000000, 1, 62, 1, 1, 1, 0),
    RK3036_PLL_RATE(1464000000, 1, 61, 1, 1, 1, 0),
    RK3036_PLL_RATE(1440000000, 1, 60, 1, 1, 1, 0),
    RK3036_PLL_RATE(1416000000, 1, 59, 1, 1, 1, 0),
    RK3036_PLL_RATE(1392000000, 1, 58, 1, 1, 1, 0),
    RK3036_PLL_RATE(1368000000, 1, 57, 1, 1, 1, 0),
    RK3036_PLL_RATE(1344000000, 1, 56, 1, 1, 1, 0),
    RK3036_PLL_RATE(1320000000, 1, 55, 1, 1, 1, 0),
    RK3036_PLL_RATE(1296000000, 1, 54, 1, 1, 1, 0),
    RK3036_PLL_RATE(1272000000, 1, 53, 1, 1, 1, 0),
```

```

RK3036_PLL_RATE(1248000000, 1, 52, 1, 1, 1, 0),
RK3036_PLL_RATE(1200000000, 1, 50, 1, 1, 1, 0),
RK3036_PLL_RATE(1188000000, 2, 99, 1, 1, 1, 0),
RK3036_PLL_RATE(1104000000, 1, 46, 1, 1, 1, 0),
RK3036_PLL_RATE(1100000000, 12, 550, 1, 1, 1, 0),
RK3036_PLL_RATE(1008000000, 1, 84, 2, 1, 1, 0),
RK3036_PLL_RATE(1000000000, 1, 125, 3, 1, 1, 0),
RK3036_PLL_RATE( 984000000, 1, 82, 2, 1, 1, 0),
RK3036_PLL_RATE( 960000000, 1, 80, 2, 1, 1, 0),
RK3036_PLL_RATE( 936000000, 1, 78, 2, 1, 1, 0),
RK3036_PLL_RATE( 912000000, 1, 76, 2, 1, 1, 0),
RK3036_PLL_RATE( 900000000, 4, 300, 2, 1, 1, 0),
RK3036_PLL_RATE( 888000000, 1, 74, 2, 1, 1, 0),
RK3036_PLL_RATE( 864000000, 1, 72, 2, 1, 1, 0),
RK3036_PLL_RATE( 840000000, 1, 70, 2, 1, 1, 0),
RK3036_PLL_RATE( 816000000, 1, 68, 2, 1, 1, 0),
RK3036_PLL_RATE( 800000000, 1, 100, 3, 1, 1, 0),
RK3036_PLL_RATE( 700000000, 6, 350, 2, 1, 1, 0),
RK3036_PLL_RATE( 696000000, 1, 58, 2, 1, 1, 0),
RK3036_PLL_RATE( 676000000, 3, 169, 2, 1, 1, 0),
RK3036_PLL_RATE( 600000000, 1, 75, 3, 1, 1, 0),
RK3036_PLL_RATE( 594000000, 1, 99, 4, 1, 1, 0),
RK3036_PLL_RATE( 533250000, 8, 711, 4, 1, 1, 0),
RK3036_PLL_RATE( 504000000, 1, 63, 3, 1, 1, 0),
RK3036_PLL_RATE( 500000000, 6, 250, 2, 1, 1, 0),
RK3036_PLL_RATE( 408000000, 1, 68, 2, 2, 1, 0),
RK3036_PLL_RATE( 312000000, 1, 52, 2, 2, 1, 0),
RK3036_PLL_RATE( 297000000, 1, 99, 4, 2, 1, 0),
RK3036_PLL_RATE( 216000000, 1, 72, 4, 2, 1, 0),
RK3036_PLL_RATE( 148500000, 1, 99, 4, 4, 1, 0),
RK3036_PLL_RATE( 106500000, 1, 71, 4, 4, 1, 0),
RK3036_PLL_RATE(  96000000, 1, 64, 4, 4, 1, 0),
RK3036_PLL_RATE(  74250000, 2, 99, 4, 4, 1, 0),
RK3036_PLL_RATE(  65000000, 1, 65, 6, 4, 1, 0),
RK3036_PLL_RATE(  54000000, 1, 54, 6, 4, 1, 0),
RK3036_PLL_RATE(  27000000, 1, 27, 6, 4, 1, 0),
{ /* sentinel */ },
};

```

3.1.2 PLL Calculation Formula

```

VCO = 24M * FBDIV / REFDIV (450M ~ 2200M)
/*The larger the VCO, the smaller the jitter, and the higher the power consumption; the
smaller the REFDIV, the shorter the PLL LOCK time*/
FOUT = VCO / POSTDIV1 / POSTDIV2 /
/* POSTDIV1 >= POSTDIV2*/

```

For example: $VCO = 24M * 99 / 2 = 1188M$

$FOUT = 1188 / 2 / 1 = 594M$

If additional PLL frequencies are required, fill in the table according to the above formula.

There is a special type of PLL that cannot be found in the table and will automatically calculate the PLL parameters. For example:

(Note: However, when using automatic calculation, VCO cannot be guaranteed to be as large as possible, so it is not recommended to use if there are jitter requirements for the PLL.)

3.2 Configuration of Specific Clocks

3.2.1 LCDC Display-Related Clocks

The DCLK of LCDC is determined by the resolution of the current screen, hence there can be significant differences between different products. Therefore, on the RK platform, LCDC's DCLK generally occupies a unique PLL. Since a unique PLL is required, the frequency of this PLL will vary according to the screen requirements. Generally, this PLL is expected to automatically calculate PLL parameters. Additionally, other clocks that have specific clock requirements should avoid being connected to this PLL. As shown in the table below:

Product Name	PLL
RK303X	Exclusive CPLL
RK312X	Exclusive CPLL
RK322X	Exclusive HDMIPHY PLL
RK3288X	Exclusive CPLL
RK3368	Exclusive NPLL
RK3399	Supports Dual Display with Exclusive CPLL and VPLL
RK3568	Supports 3 Ports, Exclusive HPLL and VPLL
RK3588	Supports 4 Ports, Exclusive V0PLL, and 2 HDMIPHYPLLs
RK3576	Supports 3 Ports, Exclusive VPLL, and 1 HDMIPHYPLL
RK3506	Supports 1 Port, Nearest Frequency Division

Figure 3-1 DCLK Allocation Diagram

For the setting of display CLOCK, there are significant differences across platforms. Here, RK322X and RK3288 are used as examples.

RK322X:

HDMIPHY PLL is used to provide DCLK for LCDC, so it is relatively simple. The DCLK LCDC requires as much as HDMIPHY outputs, which is the phase-locked output implemented internally by HDMIPHY.

RK3288:

Figure 3-2: Schematic diagram of fractional frequency division clocks

3.2.3 Ethernet Clock

For Ethernet clocks, precision is generally required. Gigabit Ethernet demands a precise frequency of 50M, while 10 Gigabit Ethernet requires a precise frequency of 125M. Typically, for devices with Ethernet requirements, the PLL must also output a precise clock. If the current clock scheme cannot provide a precise clock due to other reasons, Ethernet must use an external clock crystal. This is determined based on project requirements and actual product solutions.

3.2.4 PLL Parameters for Multi-Path Clock Input

The reference clock for RK3576 and RK3506 PLL supports both original crystal input and newly added IO input. It is primarily used in conjunction with audio functions for synchronization purposes. For detailed usage methods, please contact our support.